Clean software development

Day 2

Alessandro Sciarra

Z02 – Software Development Center

7 October 2025

GOETHE UNIVERSITÄT FRANKFURT AM MAIN

AxelKrypton

CRC-TR 211
Strong-interaction matter under extreme conditions

HGS-HIRe for FAIR
Helmholtz Graduate School for Hadron and Ion Research

Topics of the day

1. Clean code principles
   - Meaningful names
   - Comments and formatting
   - Documentation
   - Functions and classes
   - IOSP
2. Clean testing (II)
3. The time has come
4. Clean testing (III)
5. Tests, tests, tests
6. General good practices
7. Refactor!
8. Sum up and closing

An enlightening day

Today you will learn and/or reflect on simple language-independent ideas and principles which will be the starting point of a new software era in your daily life!

Clean code principles

Meaningful names

Use intention-revealing names

Why is it hard to tell what this code is doing?

```
using myVecVec = std::vector<std::vector<int>>;

myVecVec FC(const myVecVec& b)
{
    myVecVec l;
    for(const std::vector<int>& c : b){
        if(c[0] == 4)
            l.push_back(c);
    }
    return l;
}
```

Use intention-revealing names

Why is it hard to tell what this code is doing?

```
using myVecVec = std::vector<std::vector<int>>;

myVecVec FC(const myVecVec& b)
{
    myVecVec l;
    for(const std::vector<int>& c : b){
        if(c[0] == 4)
            l.push_back(c);
    }
    return l;
}
```

- What kind of object is contained in `l`?
- What does `0` represent in the `if`-clause?
- What does `4` mean in the `if`-clause?
- How would I use the object being returned?

Use intention-revealing names

Good names are crucial for readability!

```
using myVecVec = std::vector<std::vector<int>>;

myVecVec getFlaggedCells(const myVecVec& gameBoard)
{
    myVecVec flaggedCells;
    for(const std::vector<int>& cell : gameBoard){
        if(cell[STATUS_VALUE] == FLAGGED)
            flaggedCells.push_back(cell);
    }
    return flaggedCells;
}
```

- What kind of object is contained in `flaggedCells`?
- What does `STATUS_VALUE` represent in the `if`-clause?
- What does `FLAGGED` mean in the `if`-clause?
- How would I use the object being returned?

Use intention-revealing names

Good names are crucial for readability!

```
using SetOfCells = std::vector<std::vector<int>>;

SetOfCells getFlaggedCells(const SetOfCells& gameBoard)
{
    SetOfCells flaggedCells;
    for(const Cell& cell : gameBoard){
        if(cell.isFlagged())
            flaggedCells.push_back(cell);
    }
    return flaggedCells;
}
```

- What kind of object is contained in `flaggedCells`?
- What does `STATUS_VALUE` represent in the `if`-clause?
- What does `FLAGGED` mean in the `if`-clause?
- How would I use the object being returned?

Use names which can be searched

Rule of thumb

The length of a name should be chosen w.r.t. to the size of its scope

Variables: Proportionally to their scope size

Functions: In inverse proportion to their scope (cf. C++ STL)

- Single-letter names **only** for local variables inside **short scopes**
- Single-word names **a must** for functions in widely used **libraries**
- IDE very good at renaming variables $\Rightarrow$ rename, rename, rename!

Use names which can be searched

Rule of thumb

The length of a name should be chosen w.r.t. to the size of its scope

Variables: Proportionally to their scope size

Functions: In inverse proportion to their scope (cf. C++ STL)

Something like this...

```
int s = 0;
for(int j = 0; j < 34; j++)
    s += t[j]*4/5;
```


Use names which can be searched

Rule of thumb

The length of a name should be chosen w.r.t. to the size of its scope

Variables: Proportionally to their scope size

Functions: In inverse proportion to their scope (cf. C++ STL)

...or something like this?

```
const int effective_days_per_week = 4;
const double factor = effective_days_per_week /
                      WORKING_DAYS_PER_WEEK;
int total_estimate_in_weeks = 0;
// Note that the variable 'j' was not renamed!
for(int j = 0; j < NUMBER_OF_TASKS; j++)
{
    total_estimate_in_weeks += factor * task_estimate_in_days[j];
}
```


Other important principles

- Avoid disinformation
- Use pronounceable names → `timestamp` better than `ymdhms`
- Avoid mental mapping
- Pick one word per concept → `get`, `fetch`, `retrieve`, `obtain`
- Do not be cute
- …

Take home lesson

Choose carefully every name!

↓

Rename when you find better ones!

Comments and formatting

Do not comment bad code, rewrite it

Comments are an **unfortunate necessity**,
NOT a great achievement!

— Robert C. Martin —

Do not comment bad code, rewrite it

Avoid comments!

The proper use of comments is to compensate for our failure to express ourself in code. Note that I used the word failure. I meant it. Comments are always failures. [...] Why am I so down on comments? Because they lie. Not always, and not intentionally, but too often. The older a comment is, and the farther away it is from the code it describes, the more likely it is to be just plain wrong.

R. C. Martin

- To keep comments up-to-date costs energy and this energy should rather go into make the code clearer and more expressive!
- Inaccurate comments are far worse than no comments!
- Truth can only be found in one place: the code.

Explain yourself in the code

What you should strive to avoid...

```
Person e; // The employee we are considering

// Check if the employee is eligible for full benefits
if( (e.flags & HOURLY_FLAG) && e.age > 65 )
```

Explain yourself in the code

What you should strive to avoid...

```
Person e; // The employee we are considering

// Check if the employee is eligible for full benefits
if( (e.flags & HOURLY_FLAG) && e.age > 65 )
```

...in favour of expressive code

```
Person employee;

if(employee.isEligibleForFullBenefits())
```


Explain yourself in the code

What you should strive to avoid...

```
Person e; // The employee we are considering

// Check if the employee is eligible for full benefits
if( (e.flags & HOURLY_FLAG) && e.age > 65 )
```

...in favour of expressive code

```
Person employee;

if(employee.isEligibleForFullBenefits())
```

- It is simple to come up with good ideas
- It is often simply matter of creating a function whose name expresses the content of the comment!

Good comments

But try to limit them as much as possible!

Informative comment

```
// Format matched:      hh:mm:ss dd Month yyyy
std::regex timePattern("(\\d{2}:){2}\\d{2}, \\d{2} \\S+ \\d{4}");
```

Good comments

But try to limit them as much as possible!

Informative comment

```
// Format matched:      hh:mm:ss dd Month yyyy
std::regex timePattern("(\\d{2}:){2}\\d{2}, \\d{2} \\S+ \\d{4}");
```

Explanation of intent

```
// Checking the residuum possibly not every
// iteration allows for speed up on GPUs
if(iterationNumber % CG_CHECK_RESIDUUM_EVERY == 0)
```

Good comments

But try to limit them as much as possible!

Informative comment

```
// Format matched:      hh:mm:ss dd Month yyyy
std::regex timePattern("(\\d{2}:){2}\\d{2}, \\d{2} \\S+ \\d{4}");
```

Explanation of intent

```
// Checking the residuum possibly not every
// iteration allows for speed up on GPUs
if(iterationNumber % CG_CHECK_RESIDUUM_EVERY == 0)
```

Clarification when using cryptic external API

```
// Check if root was found up to desired precision
if(gsl_fcmp(functionValue, 0.0, epsilon) == 0)
```


Good comments

But try to limit them as much as possible!

- Informative comment
- Explanation of intent, Clarification

Amplification

```
// See https://stackoverflow.com/a/1736040 for more information.
template<typename T>
std::string getDefaultForHelper(const T& number)
```

```
// Removing trailing spaces is crucial since any would break the
// following algorithm. It might leave the string unchanged, but
// the absence of trailing spaces is at the moment not guaranteed!
boost::trim_right(keyToDecipher);
```


Good comments

But try to limit them as much as possible!

- Informative comment
- Explanation of intent, Clarification
- Amplification

TODO comments

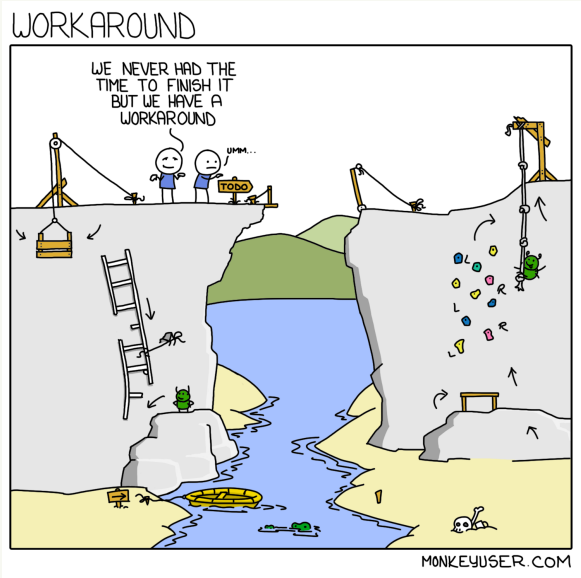
```
// TODO: The following two classes should be merged into one!
```

Legal comments

```
/*****************************************
 *  Copyright (c) 2018 Alessandro Sciarra  *
 *                                         *
 *  This file is part of BaHaMAS.          *
 *****************************************/
```

Good comments

But try to limit them as much as possible!



- Informative comment
- Explanation of intent, Clarification
- Amplification
- TODO comments → border line: really needed in a code? Do not check them in!
- Legal comments → border line: not legally required & overlap with version control.

Automatic documentation comments

If you are writing a public API, then you should certainly provide a good documentation. But be aware that documentation comments can be as misleading, misplaced and lying as any other comment.

Bad comments

Several bad habits in one example

```
/**
* @param longTitle The complete title of the CD
* @param longAuthor The complete author of the CD
* @param nTracks The number of tracks of the CD
* @param durationInMinutes The duration of the CD in minutes
*/
void Jukebox::addCD(std::string longTitle,
                    std::string longAuthor,
                    /*int nTracks,*/ int durationInMinutes)
{
    // Create new CD to be later added to the list
    CD cd;
    cd.title    = longTitle;
    cd.author   = longAuthor;
    //cd.tracks   = nTracks;
    cd.duration = durationInMinutes;
    if(cd.duration<60) // Added by James
       throw std::invalid_argument();
    cdList.push_back(cd);
} // addCD method
```


Bad comments

The same code cleaned up

```
void Jukebox::addCD(std::string longTitle,
                    std::string longAuthor,
                    int durationInMinutes)
{
    CD cd;
    cd.title    = longTitle;
    cd.author   = longAuthor;
    cd.duration = durationInMinutes;
    if(cd.duration<60)
        throw std::invalid_argument();
    cdList.push_back(cd);
}
```

The code becomes smaller and it reads well!

Why should you add a comment here?!

Bad comments

- Redundant comments → Avoid
- Noise comments → Avoid
- Misleading comments → Avoid
- Comments instead of a good name → Use good name
- Closing brace comments → Rarely
- Attribution comments → Delete, use version control
- Commented-out code → Delete, use version control
- Unclear, cryptic comments → Remove or clarify

Have **good reasons** before adding a comment!

Are they?

Formatting

Yes, formatting is important!

The 3 most important rules:

Formatting

Yes, formatting is important!

The 3 most important rules:

1. Stay coherent with what you find

Formatting

Yes, formatting is important!

The 3 most important rules:

1. Stay coherent with what you find
2. Stay coherent with what you find

Formatting

Yes, formatting is important!

The 3 most important rules:

1. Stay coherent with what you find
2. Stay coherent with what you find
3. Stay coherent with what you find

This starts with but it is not limited to formatting!

Especially important when working on an existing codebase!

Formatting

Yes, formatting is important!

- How big should be a file?
  → Try to stay below few hundreds, the reader will be grateful!
- Give some vertical breath to your code
- Care about length of lines (80 to 140 characters)
  → Crucial if the code is readable from browser (e.g. GitHub)
- Consider horizontal alignment and spacing (e.g. around operators)
- Be coherent (e.g. braces)

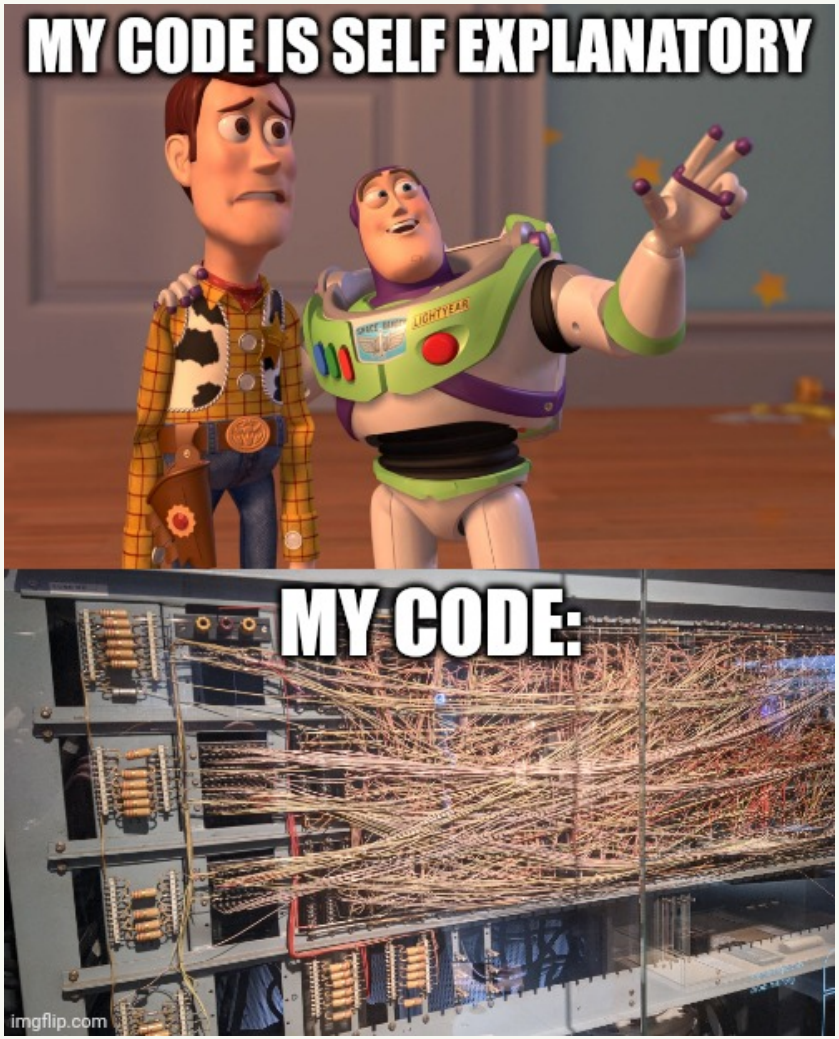
How to work when different people write in the same code base?

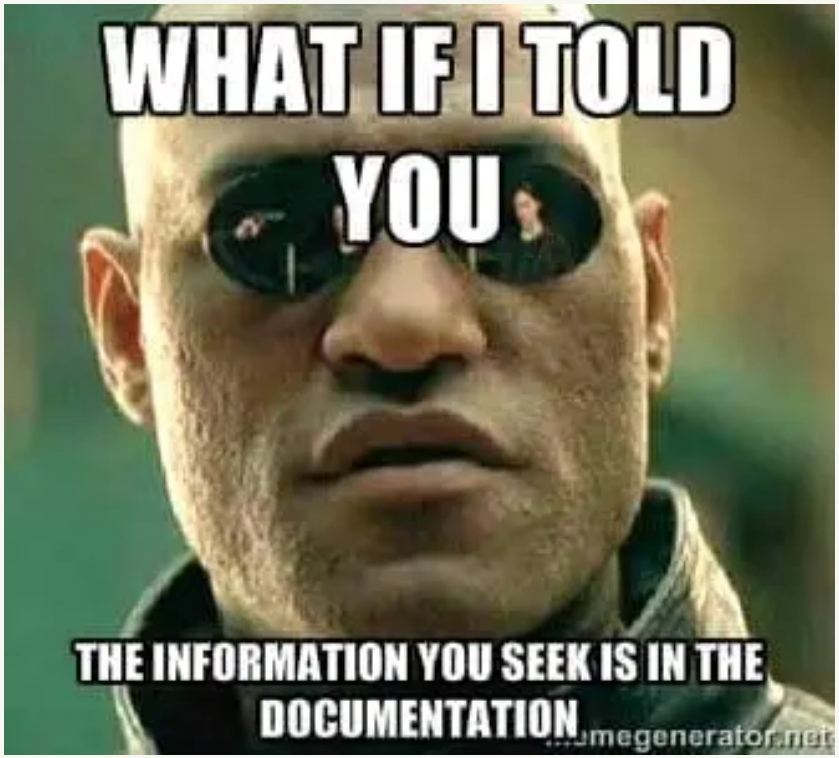
Agree on a style and use a tool (e.g. clang-format) to enforce it

Documentation

Some tension

Some tension





Some tension

Never forget it!

Software testing

A professionalized discipline integrated into the software development process.

Riona MacNamara, Google

Never forget it!

Software testing

A professionalized discipline integrated into the software development process.

Riona MacNamara, Google

Software documentation

A professionalized discipline integrated into the software development process

Never forget it!

It is about culture and mind setup

1. Writing documentation belongs to code development!
2. Code changes can invalidate documentation!
3. Good testing code documents your codebase

Never forget it!

It is about culture and mind setup

1. Writing documentation belongs to code development!
2. Code changes can invalidate documentation!
3. Good testing code documents your codebase

Writing documentation is boring and people do not read it!

That's why **you need to find the best compromise for your project**.

What's the best approach? What's your documentation for?
Is that detail needed to be documented?

Choose your tool carefully

Sphinx

Doxygen

Mkdocs

Docsify

...

The list of tools is longer than you might think

There are even lists of tools available!

It is all about the mind setup!

A software engineer

[...] many overestimate the cost of maintaining documentation. If you have a system in place, incremental improvements to docs don't have to be a huge burden that suck time away from other engineering tasks.

Another software engineer

And underestimate the damage that inaccurate docs have on users' opinions. Once a user discovers inaccurate content, they are leery of trusting anything else you say.

Functions and classes

Common general aspects

SMALL

Do one thing. Do it well. Do it only!

- Functions and classes interfaces should stay small (see next slide)
- Classes should fulfil the single responsibility principle
- One level of abstraction per function

Common general aspects

SMALL

Do one thing. Do it well. Do it only!

- Functions and classes interfaces should stay small (see next slide)
- Classes should fulfil the single responsibility principle
- One level of abstraction per function

Single-responsibility class

```
class Version {
  public:
    int majorNumber();
    int minorNumber();
    int buildNumber();
  private:
    // [...]
};
```


Common general aspects

SMALL

Do one thing. Do it well. Do it only!

- Functions and classes interfaces should stay small (see next slide)
- Classes should fulfil the single responsibility principle
- One level of abstraction per function

1. Let's start with an example of bad code
2. and then have a look to how it should be

Common general aspects

SMALL

Do one thing. Do it well. Do it only!

Impolite, rude code → up and down to understand it!

```
void Machine::MakeCoffee(const Coffee& typeOfCoffee){
    auto neededT = temperature(typeOfCoffee);
    if(neededT < 100)
        warmUp(neededT);
    while(grinder_ != nullptr)
        prepareGrinder();
    grind(calculateGrams(typeOfCoffee));
    auto neededP = pressure(typeOfCoffee);
    prepareToBrew(neededT, neededP);
    if(status_ == READY_TO_BREW)
        brew(typeOfCoffee);
    else
        throw std::runtime_error("Unable to brew!");
}
```


Common general aspects

SMALL

Do one thing. Do it well. Do it only!

- Functions and classes interfaces should stay small (see next slide)
- Classes should fulfil the single responsibility principle
- One level of abstraction per function

Single-level-of-abstraction function

```
void Machine::MakeCoffee(const Coffee& typeOfCoffee){
    WarmUpMachineIfNeeded(typeOfCoffee);
    GrindCoffee(typeOfCoffee);
    SetPressureAndTemperature(typeOfCoffee);
    BrewCoffee(typeOfCoffee);
}
```


More about this one-thing idea

> How big should a function be?
>
> Robert C. Martin

More about this one-thing idea

Back to our example

```
void Machine::MakeCoffee(const Coffee& typeOfCoffee){
    // 1. WarmUpMachineIfNeeded
    auto neededT = temperature(typeOfCoffee);
    if(neededT < 100)
        warmUp(neededT);
    // 2. GrindCoffee
    while(grinder_ != nullptr)
        prepareGrinder();
    grind(calculateGrams(typeOfCoffee));
    // 3. SetPressureAndTemperature, needs T again
    auto neededP = pressure(typeOfCoffee);
    prepareToBrew(neededT, neededP);
    // 4. BrewCoffee
    if(status_ == READY_TO_BREW)
        brew(typeOfCoffee);
    else
        throw std::runtime_error("Unable to brew!");
}
```

More about this one-thing idea

How big should a function be?

A function should do one thing. But what's one thing?
A function does one thing, if you cannot meaningfully extract another function from it.

Robert C. Martin

Result after extraction

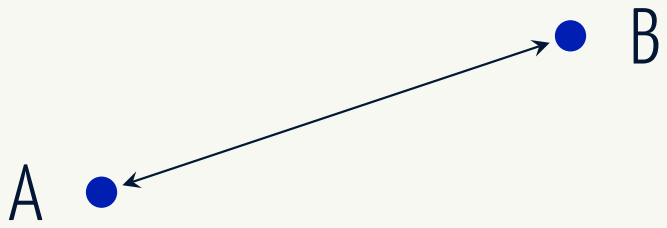
```
void Machine::MakeCoffee(const Coffee& typeOfCoffee){
    WarmUpMachineIfNeeded(typeOfCoffee);
    GrindCoffee(typeOfCoffee);
    SetPressureAndTemperature(typeOfCoffee);
    BrewCoffee(typeOfCoffee);
}
```


Limit the number of arguments

Make useful abstractions whenever it helps

When you call the function, which argument is what?

```
double Distance(double, double, double, double);
double Distance(double, double, double, double, double, double);
```

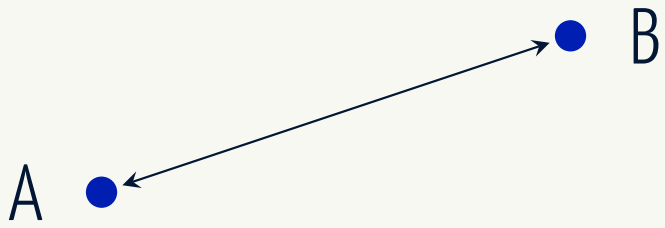


Limit the number of arguments

Make useful abstractions whenever it helps

When you call the function, which argument is what?

```
double Distance(double, double, double, double);
double Distance(double, double, double, double, double, double);
```



Isn't this much better?

```
struct Point2D {
    double x, y;
};
```

```
struct Point3D {
    double x, y, z;
};
```

```
double Distance(std::pair<Point2D, Point2D>);
double Distance(std::pair<Point3D, Point3D>);
```

IOSP

Integration Operation Segregation Principle

Clear separation of:

Operation: A function which contains exclusively logic, meaning transformations, control structures or API invocations.

Integration: A function which does not contain any logic but exclusively calls other functions of the code base.

Operation

```
bool isPrime(unsigned int number)
{
    if(number == 1) return false;
    for(auto i = 2u; i <= std::sqrt(number); ++i)
    {
        if(number%i == 0) return false;
    }
    return true;
}
```


Integration Operation Segregation Principle

Clear separation of:

Operation: A function which contains exclusively logic, meaning transformations, control structures or API invocations.

Integration: A function which does not contain any logic but exclusively calls other functions of the code base.

Integration

```
int main(int argc, char *argv[])
{
    WelcomeUserToTheGame();
    Minesweeper minesweeper(argc, argv);
    minesweeper.playGame();
    PrintGameResult();
}
```


Clean testing (II)

The curse of the **unit** word

What is a **unit** in unit testing?

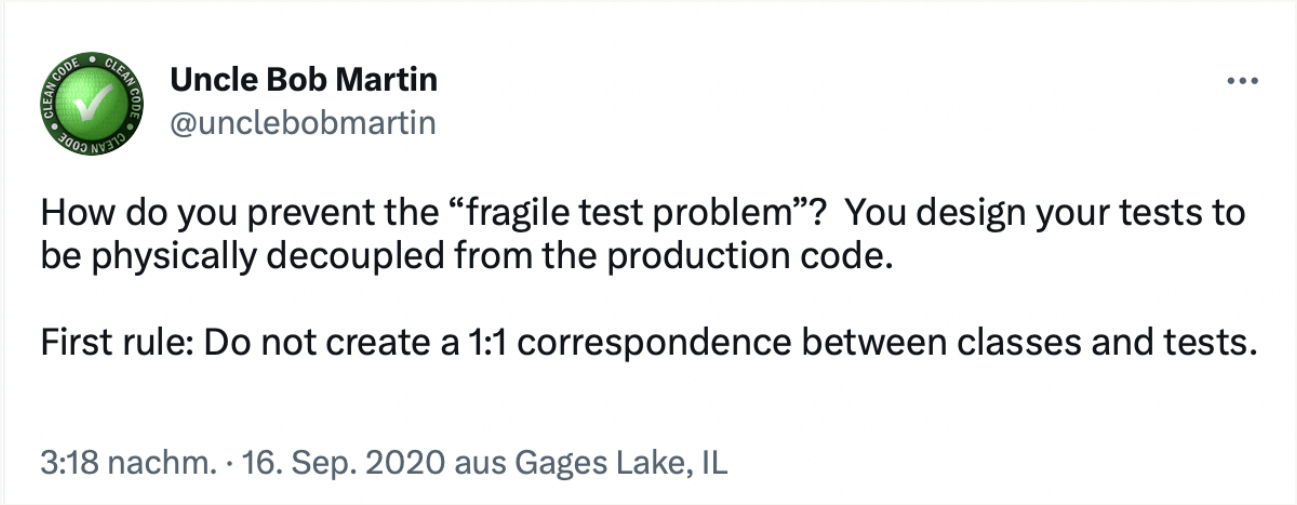
> A group of related functions often called a module.
>
> — Wikipedia

> A unit test should test a requirement, only test against the stable contract of the API.
>
> — Ian Cooper

A unit is not necessarily a class or a function

This is possibly one of the most misunderstood concepts in software engineering in the last decade. Sometimes a unit is a class or a function, but there should be a good reason for such a correspondence. **More often a unit should be (the public API of) a module.**

What is a **unit** in unit testing?



Uncle Bob Martin
@unclebobmartin

How do you prevent the “fragile test problem”? You design your tests to be physically decoupled from the production code.

First rule: Do not create a 1:1 correspondence between classes and tests.

3:18 nachm. · 16. Sep. 2020 aus Gages Lake, IL

A unit is not necessarily a class or a function

This is possibly one of the most misunderstood concepts in software engineering in the last decade. Sometimes a unit is a class or a function, but there should be a good reason for such a correspondence. **More often a unit should be (the public API of) a module.**

Wait, what's wrong
in testing each single class?

It depends

> **TDD, Where Did It All Go Wrong**
>
> A couple of times we were very proud of the fact that we produced some code that went to production with almost zero defects. [...] I looked at this test suite. One things I noticed was that we had about three times as much test code as production code.
>
> Ian Cooper

- Some classes are simply implementation details
  - ↳ You would like to freely change them
- Imagine a module with few classes working together { in your implementation, today }
  - ↳ Does it make sense to mock all but one several times?
- If a class happens to be the API of a module, probably that's a unit

White- and black-box testing!?

I'll test a single class next in examples,
but you now know what a **unit** is!

White box testing

- You allow yourself in using all code in your tests
  - ↳ even private methods and members
  - ↳ implementation details are used to test interface

White box testing

- You allow yourself in using all code in your tests
- In some languages (e.g. Python) easier than in others (e.g. C++)

Consider this class

```
// File Cup.h

class Cup {
    public:
    Cup();
    bool IsEmpty();
    bool Drink();
    bool Fill();
    private:
    bool isEmpty_ = true;
};
```

Test file

```
#include "vir/test.h"
#include "Cup.h"


TEST(Cup::Cup) {
    Cup cup{};
    VERIFY(cup.isEmpty_);
}
TEST(Cup::IsEmpty) { /*...*/ }
TEST(Cup::Fill)    { /*...*/ }
TEST(Cup::Drink)   { /*...*/ }
```

This does not compile!

I took this example from Dave Steffen's talk at CppCon2020.

White box testing

- You allow yourself in using all code in your tests
- In some languages (e.g. Python) easier than in others (e.g. C++)

Consider this class

```
// File Cup.h

class Cup {
    public:
    Cup();
    bool IsEmpty();
    bool Drink();
    bool Fill();
    private:
    bool isEmpty_ = true;
};
```

Test file

```
#include "vir/test.h"
#include "Cup.h"
#define private public // UB!!!

TEST(Cup::Cup) {
    Cup cup{};
    VERIFY(cup.isEmpty_);
}
TEST(Cup::IsEmpty) { /*...*/ }
TEST(Cup::Fill)    { /*...*/ }
TEST(Cup::Drink)   { /*...*/ }
```

This relies on undefined behaviour!!!

I took this example from Dave Steffen's talk at CppCon2020.

White box testing

- You allow yourself in using all code in your tests
- In some languages (e.g. Python) easier than in others (e.g. C++)

Consider this class

```
// File Cup.h

class Cup {
    public:
    Cup();
    bool IsEmpty();
    bool Drink();
    bool Fill();
    private:
    friend struct CupTester;
    bool isEmpty_ = true;
};
```

Test file

```
#include "vir/test.h"
#include "Cup.h"
struct CupTester { /*...*/ };

TEST(Cup::Cup) {
    Cup cup{};
    CupTester tester{cup};
    VERIFY(tester.isEmpty_);
}
TEST(Cup::IsEmpty) { /*...*/ }
TEST(Cup::Fill)    { /*...*/ }
TEST(Cup::Drink)   { /*...*/ }
```


I took this example from Dave Steffen's talk at CppCon2020.

White box testing

- You allow yourself in using all code in your tests
- In some languages (e.g. Python) easier than in others (e.g. C++)
- Something to consider to start working on legacy code!

White box testing

- You allow yourself in using all code in your tests
- In some languages (e.g. Python) easier than in others (e.g. C++)
- Something to consider to start working on legacy code!
- What happens when you change your implementation?
  - ↳ Compilation might break
  - ↳ If not, some tests will probably fail
  - ↳ You likely need to change your tests code
  - ↳ For large refactoring, it might be boring/frustrating

Black-box testing

- What about just testing the public interface?

Consider this class

```
// File Cup.h

class Cup {
    public:
    Cup();
    bool IsEmpty();
    bool Drink();
    bool Fill();
    private:
    bool isEmpty_ = true;
};
```

Source code same as before

Test file

```
#include "vir/test.h"
#include "Cup.h"

TEST(Cup::Cup)      { /*...*/ }
TEST(Cup::IsEmpty) { /*...*/ }
TEST(Cup::Fill)     { /*...*/ }
TEST(Cup::Drink)    { /*...*/ }
```


Black-box testing

- What about just testing the public interface?

Consider this class

```
// File Cup.h

class Cup {
    public:
    Cup();
    bool IsEmpty();
    bool Drink();
    bool Fill();
    private:
    bool isEmpty_ = true;
};
```

Source code same as before

Test file

```
#include "vir/test.h"
#include "Cup.h"

TEST(Cup::Cup) {
    Cup cup{};
    VERIFY(cup.IsEmpty());
}
TEST(Cup::IsEmpty) { /*...*/ }
TEST(Cup::Fill)    { /*...*/ }
TEST(Cup::Drink)   { /*...*/ }
```


Black-box testing

- What about just testing the public interface?

Consider this class

```
// File Cup.h

class Cup {
    public:
    Cup();
    bool IsEmpty();
    bool Drink();
    bool Fill();
    private:
    bool isEmpty_ = true;
};
```

Source code same as before

Test file

```
#include "vir/test.h"
#include "Cup.h"

TEST(Cup::Cup) {
    Cup cup{};
    VERIFY(cup.IsEmpty());
}
TEST(Cup::IsEmpty){
    Cup cup{};
    VERIFY(cup.IsEmpty());
}
TEST(Cup::Fill)     { /*...*/ }
TEST(Cup::Drink)    { /*...*/ }
```

Wait, we wrote the same code twice!

Black-box testing

- What about just testing the public interface?

> **The black-box conundrum**
>
> Fundamentally if we only test via the public interface, we have a circular-logic problem: you have to use the interface to test the interface, so at some point you have to trust something which you haven't tested.
>
> Dave Steffen

Black-box testing

- What about just testing the public interface?

The black-box conundrum

Fundamentally if we only test via the public interface, we have a circular-logic problem: you have to use the interface to test the interface, so at some point you have to trust something which you haven't tested.

Dave Steffen

So, what to do then?

1. Ignore the problem { Be aware: Tests are not independent, one bug triggers many failures! }
2. Declare one member function "correct by inspection" { and start from there }
3. Abandon black-box testing as impractical

Test the behaviour, not the implementation

Do not test methods! Test behaviour!

Back to our cup example:

```
#include "vir/test.h"
#include "Cup.h"

TEST(A new cup is empty)              { /*...*/ }
TEST(An empty cup can be filled)      { /*...*/ }
TEST(A filled cup is full)            { /*...*/ }
TEST(Drinking empties a filled cup)   { /*...*/ }
```

- Test names describe the unit specifications
- Suddenly, we are completely independent from implementation details
- OK, so how does test code look like?

Do not test methods! Test behaviour!

Back to our cup example:

```
#include "vir/test.h"
#include "Cup.h"

TEST(A new cup is empty){
    Cup cup{};
    VERIFY(cup.IsEmpty());
}
TEST(An empty cup can be filled)     { /*...*/ }
TEST(A filled cup is full)           { /*...*/ }
TEST(Drinking empties a filled cup) { /*...*/ }
```


Do not test methods! Test behaviour!

Back to our cup example:

```
#include "vir/test.h"
#include "Cup.h"

TEST(A new cup is empty){
    Cup cup{};
    VERIFY(cup.IsEmpty());
}
TEST(An empty cup can be filled)     { /*...*/ }
TEST(A filled cup is full)           { /*...*/ }
TEST(Drinking empties a filled cup) { /*...*/ }
```

- Wait!! This is exactly the same code that wasn't OK few minutes ago!?
  - ↳ How is it that changing the name makes it fine?
  - ↳ How should this solve the black-box conundrum?

Do not test methods! Test behaviour!

The bottom line

We are now testing consistency of interface, not correctness of implementation!

Do not test methods! Test behaviour!

The bottom line

We are now testing consistency of interface, not correctness of implementation!

Let's say it once more:

We are now testing consistency of interface, not correctness of implementation!

Do not test methods! Test behaviour!

The bottom line

We are now testing consistency of interface, not correctness of implementation!

Let's say it once more:

We are now testing consistency of interface, not correctness of implementation!

It is not a philosophical statement!

A bug that cannot be observed under any conditions is not a bug!

Is this code correct or wrong?

```
// File Cup.h

class Cup {
    public:
    Cup()           { isEmpty_ = false; }
    bool IsEmpty()  { return !isEmpty_; }
    bool Drink()    { /*...*/ }
    bool Fill()     { /*...*/ }
    private:
    bool isEmpty_ = false;
};
```


As testing behaviour passes, the code is correct!

Is this code correct or wrong?

```
// File Cup.h

class Cup {
    public:
    Cup()           { isEmpty_ = false; }
    bool IsEmpty()  { return !isEmpty_; }
    bool Drink()    { /*...*/ }
    bool Fill()     { /*...*/ }
    private:
    bool isEmpty_ = false;
};
```

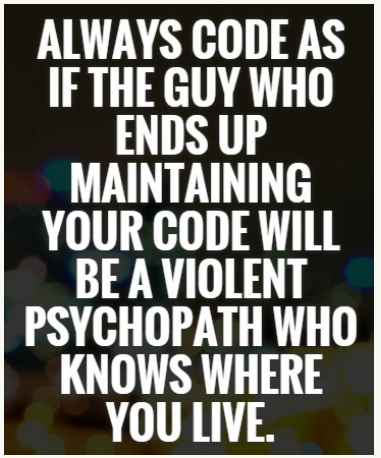
```
#include "vir/test.h"
#include "Cup.h"
TEST(A new cup is empty)              { /*...*/ }
TEST(An empty cup can be filled)      { /*...*/ }
TEST(A filled cup is full)            { /*...*/ }
TEST(Drinking empties a filled cup)   { /*...*/ }
```


Break

The time has come

A new begin

You have 90 minutes



0. You won't work on your code! → 
1. Improve names!
   ↳ Your IDE can probably rename symbols
2. Check comments → Remove or add (is there a good reason?)
3. Is there documentation? If not, how would you add some?
   ↳ Any thoughts about that?
4. Add some tests for one or more functions
5. Apply IOSP to such functions
   ↳ How do you pass around arguments?
6. Stage, commit and push your work!

Lunch break

Clean testing (III)

Tests goals – properties – good principles

General idea and overview

There is slightly different advice around, pick your favourite:

Write F.I.R.S.T tests first

Fast:	To be run frequently
Independent:	To make failures meaningful
Repeatable:	To be run everywhere with same outcome
Self-validating:	To be automatised
Timely:	To make production code testable

General idea and overview

There is slightly different advice around, pick your favourite:

Google's perspective

Readable:	Tests are correct by inspection
Correct:	Do not rely on known bugs and test real scenarios
Complete:	Test most edge cases
Documenting:	Demonstration of how the API works
Resilient:	Repeatable, independent, hard to break, hermetic, etc.

General idea and overview

There is one not-negotiable **requirement of tests**, though:

General idea and overview

There is one not-negotiable **requirement of tests**, though:

Having tests is not enough, you must trust them!

Few words about tests correctness

- Never rely on known bugs

Having tests passing is simple...

Having tests is not enough, you must trust them!

Few words about tests correctness

- Never rely on known bugs

Having tests passing is simple...

```
int cube(int x) {
    //TODO: To be implemented
    return 0;
}

TEST(cube_test) {
    VERIFY(0 == cube(2));
    VERIFY(0 == cube(3));
    VERIFY(0 == cube(42));
}
```


Having tests is not enough, you must trust them!

Few words about tests correctness

- Never rely on known bugs

...better to have them failing!

```
int cube(int x) {
    //TODO: To be implemented
    return 0;
}

TEST(cube_test) {
    VERIFY(8 == cube(2));
    VERIFY(27 == cube(3));
    VERIFY(74088 == cube(42));
}
```


Having tests is not enough, you must trust them!

Few words about tests correctness

- Never rely on known bugs
- Never mock the code that you are testing

This is probably useless

```
class MockCoffeeMachine : public CoffeeMachine {
    //For simplicity we assume always in temperature
    double warmUp() override {
        return constants::brewingT;
    }
}

BOOST_AUTO_TEST_CASE(warmUp_test) {
    MockCoffeeMachine machine{};
    BOOST_REQUIRE_CLOSE(machine.warmUp(),
    constants::brewingT, 0.1);
}
```


Having tests is not enough, you must trust them!

Few words about tests correctness

- Never rely on known bugs
- Never mock the code that you are testing

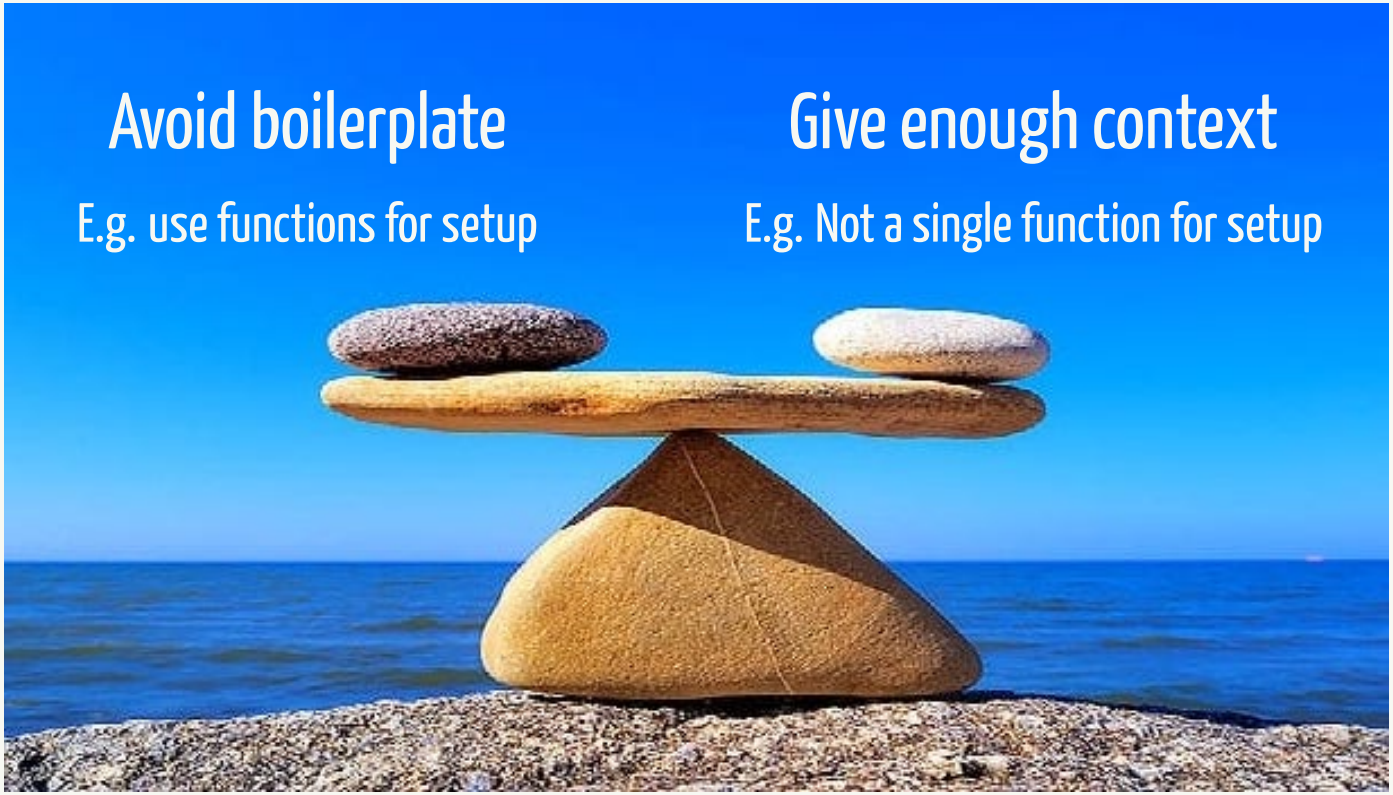
Think of a clear reaction procedure in your team

What should happen in your project when a test fails?
...and when a bug that was not caught by tests is found?

↓

Never miss any chance to improve your tests!

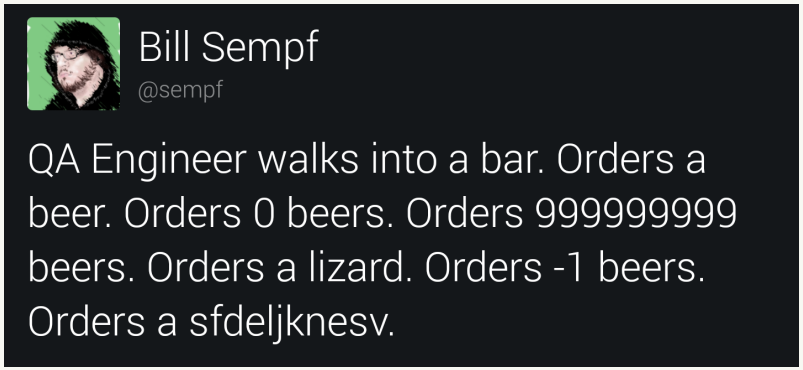
Tests readability



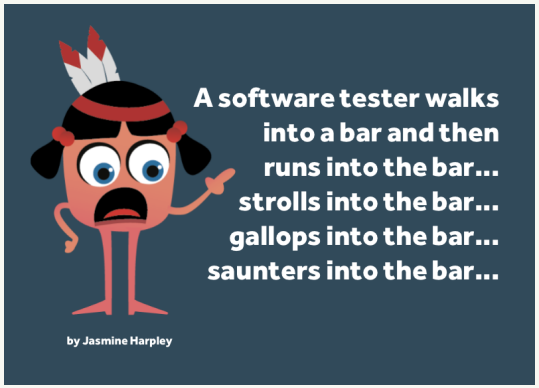
Remember: Tests are correct by inspection!

Tests completeness: What should I test and how?

- Values



- Actions



...and anything that makes sense!

Tests completeness: What should I test and how?

A very nice example from Titus Winter

```
TEST(FactorialTest, BasicTests) {
    EXPECT_EQ(1, Factorial(1));
    EXPECT_EQ(120, Factorial(5));
}
```


Tests completeness: What should I test and how?

A very nice example from Titus Winter

```
int Factorial(int n) {
    if (n == 1) return 1;
    if (n == 5) return 120;
    return -1; // TODO(goofus): figure this out.
}

TEST(FactorialTest, BasicTests) {
    EXPECT_EQ(1, Factorial(1));
    EXPECT_EQ(120, Factorial(5));
}
```


Tests completeness: What should I test and how?

A very nice example from Titus Winter

```
int Factorial(int n) {
    if (n == 1) return 1;
    if (n == 5) return 120;
    return -1; // TODO(goofus): figure this out.
}

TEST(FactorialTest, BasicTests) {
    EXPECT_EQ(1, Factorial(1));
    EXPECT_EQ(120, Factorial(5));
    EXPECT_EQ(1, Factorial(0));
    EXPECT_EQ(479001600, Factorial(12));
    EXPECT_EQ(std::numeric_limits<int>::max() , Factorial(13));
    EXPECT_EQ(1, Factorial(0));
    EXPECT_EQ(std::numeric_limits<int>::max() , Factorial(-10));
}
```

This will pretty much naturally emerge when doing TDD

The hidden value of tests

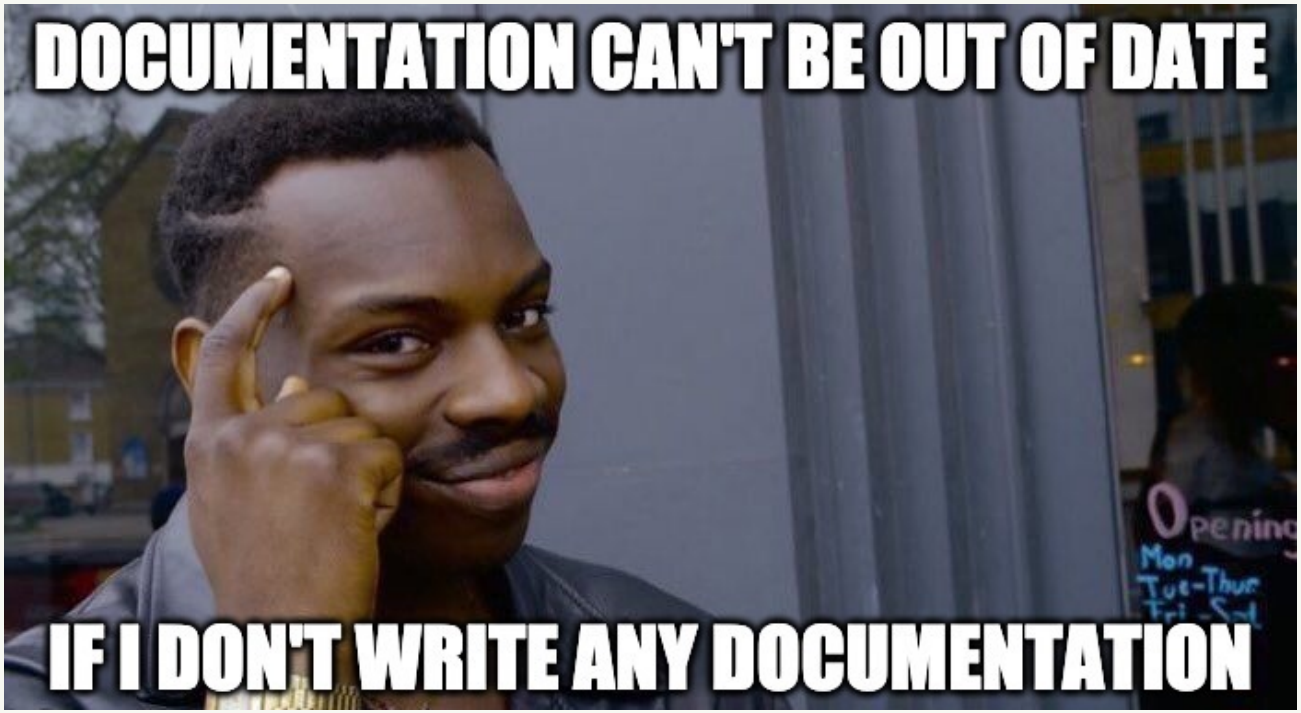
Few words about tests as API usage examples

Of course, a meme, do not take it (too) seriously, but...

The hidden value of tests

Few words about tests as API usage examples

Of course, a meme, do not take it (too) seriously, but...



...good tests should show how to use your code!

In this sense, it should be avoided to use implementation details in tests (cf. white box testing downsides).

Tests resilience: Google advice

- No flaky tests ⟶ Tests should always give the same outcome
- No brittle tests ⟶ One failure should not trigger many failures
- Tests reference values should not come from the system under test!
- Changing tests order should never change the outcome
- Tests should be as hermetic as possible ⟶ No I/O, no network, etc.
- No deep dependence ⟶ Avoid any implicit assumption

Let's see some examples!

This list and all GoogleTest examples are taken from Titus Winters' and Hyrum Wright's talk at CppCon2015.

Tests resilience: Google advice

A sneaky flaky test: What happens if the test-system is overloaded?

```
TEST(UpdaterTest, RunsFast) {
    Updater updater;
    updater.UpdateAsync();
    SleepFor(Seconds(.5)); // Half a second should be *plenty*.
    EXPECT_TRUE(updater.Updated());
}
```


This list and all GoogleTest examples are taken from Titus Winters' and Hyrum Wright's talk at CppCon2015.

Tests resilience: Google advice

A sneaky flaky test: What happens if the test-system is overloaded?

```
TEST(UpdaterTest, RunsFast) {
    Updater updater;
    updater.UpdateAsync();
    SleepFor(Seconds(.5)); // Half a second should be *plenty*.
    EXPECT_TRUE(updater.Updated());
}
```

An infamous brittle pattern

```
TEST(Logger, LogWasCalled) {
    StartLogCapture();
    EXPECT_TRUE(Frobber::Start());
    EXPECT_THAT(
      Logs(), Contains("file.cc:421: Opened file frobber.config" )
    );
}
```


This list and all GoogleTest examples are taken from Titus Winters' and Hyrum Wright's talk at CppCon2015.

Tests resilience: Google advice

Where are the reference values coming from?

```
BOOST_AUTO_TEST_CASE(Dirac_M)
{
    const hmc_float refs[4] =
        { 2610.3804893063798, 4356.332327032359,
          2614.2685771909237, 4364.1408252701831 };
    test_fermionmatrix<physics::fermionmatrix::Dslash>(refs);
}
```

Tests resilience: Google advice

Where are the reference values coming from?

```
BOOST_AUTO_TEST_CASE(Dirac_M)
{
    const hmc_float refs[4] =
        { 2610.3804893063798, 4356.332327032359,
          2614.2685771909237, 4364.1408252701831 };
    test_fermionmatrix<physics::fermionmatrix::Dslash>(refs);
}
```

A non-hermetic test: What if run twice in parallel?

```
TEST(Server, StorageTest) {
    StorageServer* server = GetStorageServerHandle();
    auto my_val = rand();
    server->Store("testkey", my_val);
    EXPECT_EQ(my_val, server->Load("testkey"));
}
```


This list and all GoogleTest examples are taken from Titus Winters' and Hyrum Wright's talk at CppCon2015.

Tests resilience: Google advice

Deep dependence example

```
class File {
    public:
    // ...
    virtual bool StatWithOptions(Stat_t* stat, StatOptions opts) {
        return this->Stat(stat); // In base class ignore options
    }
};
TEST(Filesystem, FSUsage) {
    // ... and call to StatWithOptions
    EXPECT_CALL(file, Stat(_)).Times(1);
}  // This relies on a call to base StatWithOptions!!
```

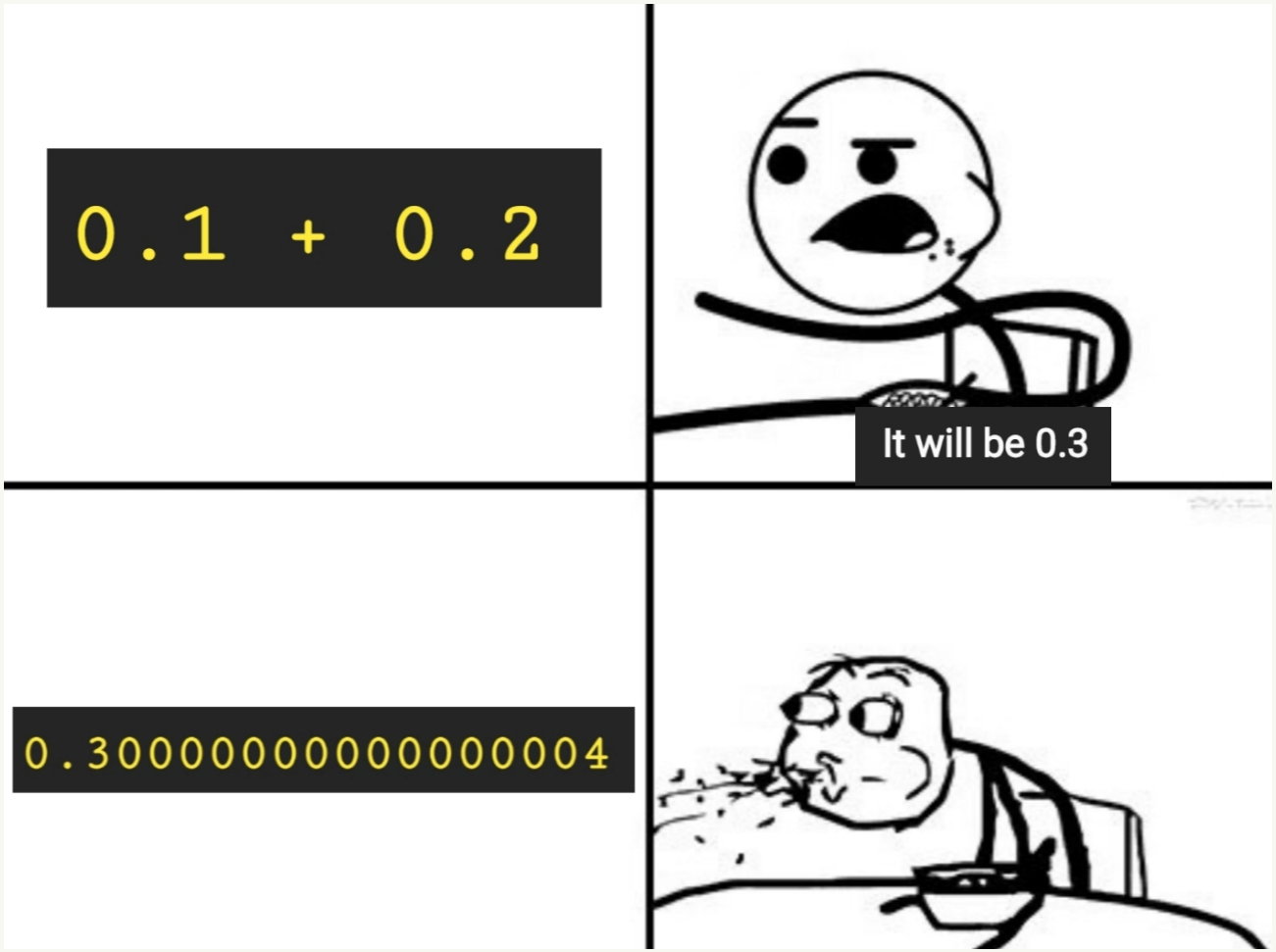
The law of implicit interfaces (AKA Hyrum's law)

Given enough users of your public interface, soon or later someone will start implicitly relying on your implementation details (speed, memory consumption, etc.).

Hyrum Wright

This list and all GoogleTest examples are taken from Titus Winters' and Hyrum Wright's talk at CppCon2015.

A slide about tests precision



A slide about tests precision

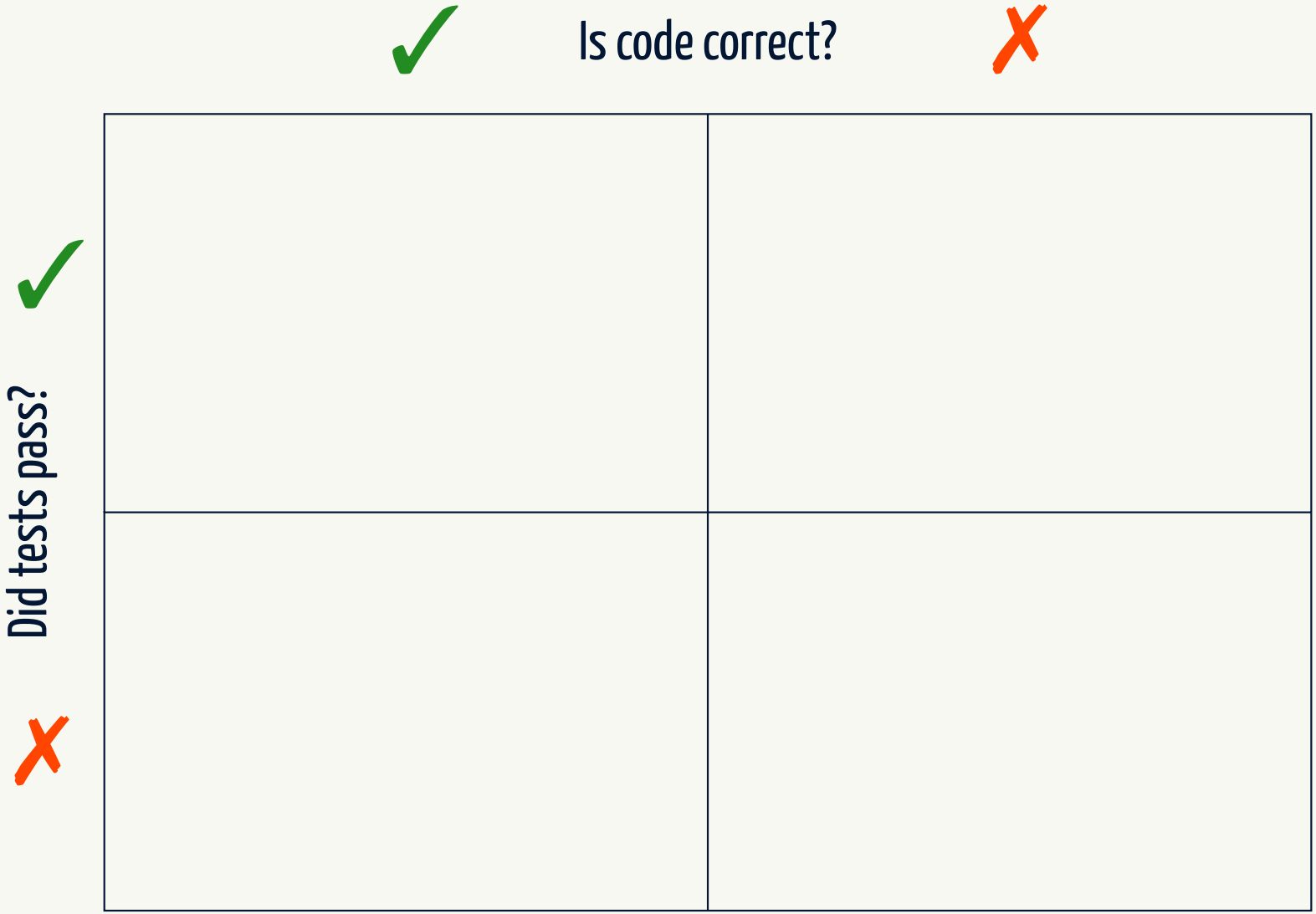
- We all know that machine precision is finite!
- Comparing floating point numbers has to be done carefully
- If you know a reference value exactly, put in the code more than the needed digits $\longrightarrow \pi = 3.14159265358979323846264 3$
- Be aware of possible compiler techniques, e.g. FMA { fused multiply-add }
  ↳ This could make your test fail if you require too high precision
- Requiring too high precision makes tests brittle
- Requiring too low precision makes tests useless
- After all it is a judgement call!

Few more words about reproducibility

How can I make my tests reproducible?

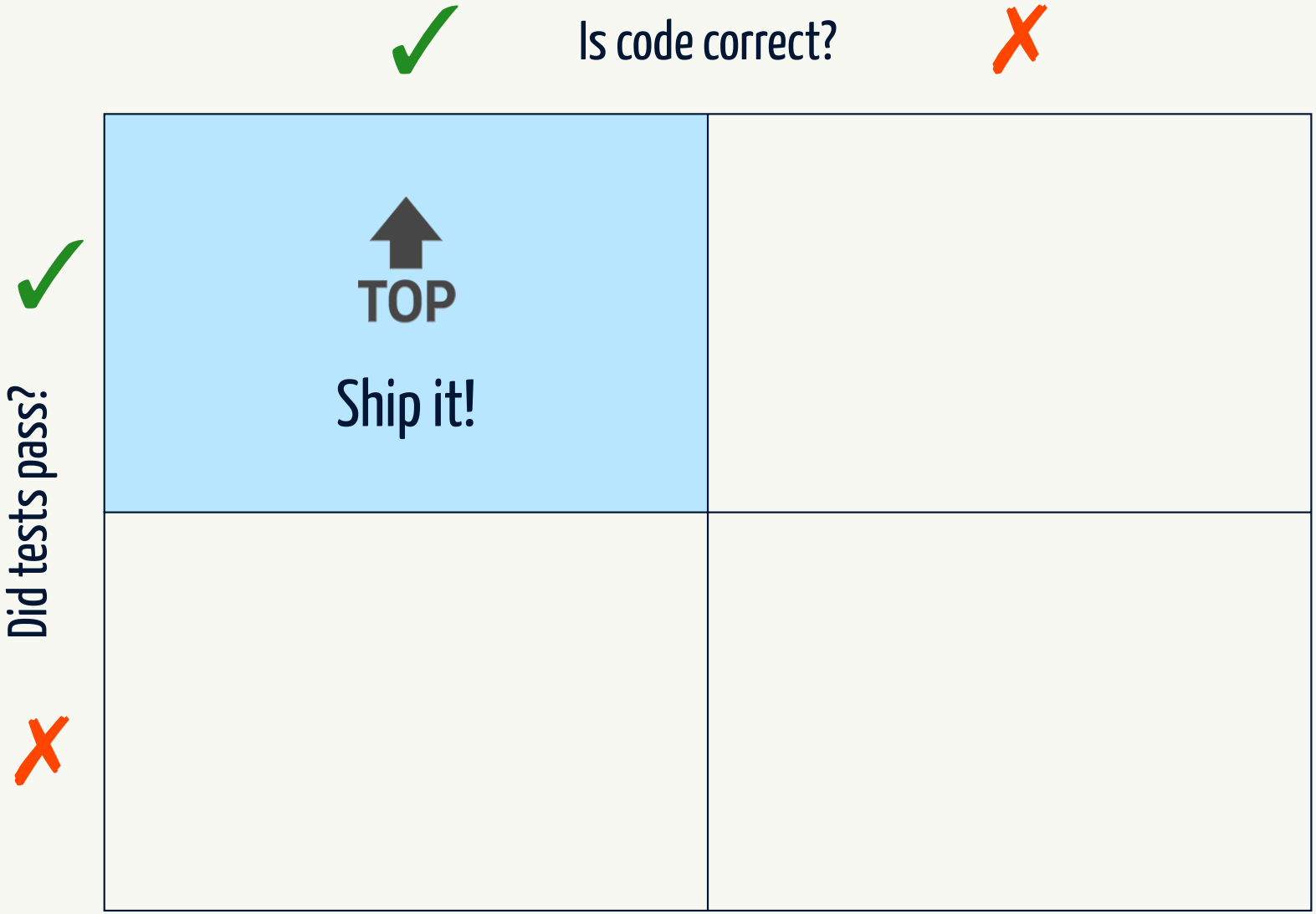
1. Isolate them from non-deterministic aspects { e.g. network, database, random numbers }
   ↳ This is not always possible!
2. Measure unwanted environment effect and subtract it in tests
3. Detect if test can be run in the present environment
   ↳ If not, either fail or handle situation
4. Handle intrinsic noise by statistics
   ↳ E.g. if random numbers are needed, measure the failure frequency and re-run the test a number of times large enough to consider a failure as such!

Tests accuracy and reacting to failures



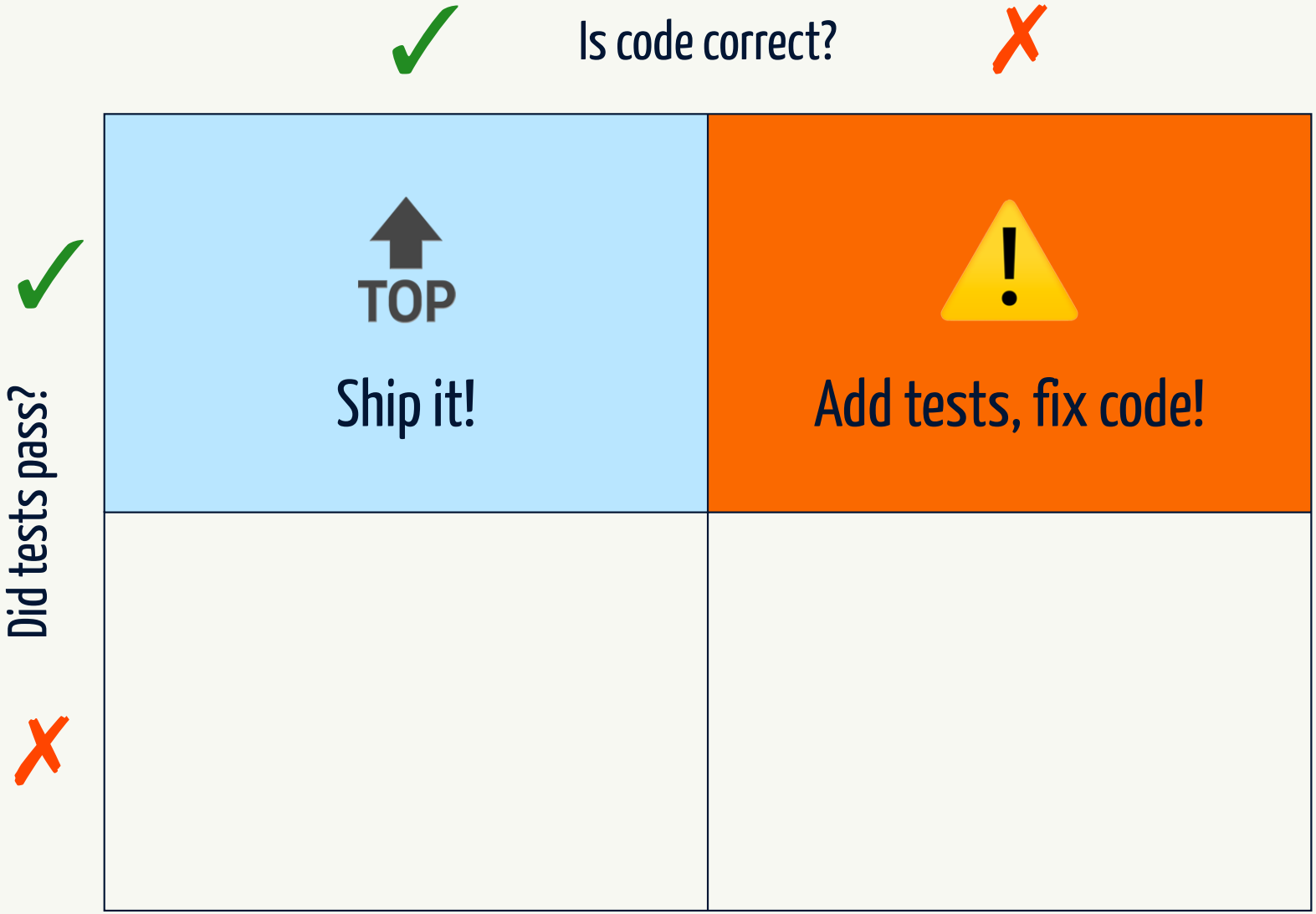
Tests accuracy means how much tests outcome matches code correctness.

Tests accuracy and reacting to failures



Tests accuracy means how much tests outcome matches code correctness.

Tests accuracy and reacting to failures



Tests accuracy means how much tests outcome matches code correctness.

Tests accuracy and reacting to failures

Did tests pass? \ Is code correct?	✓	✗
✓	 Ship it!	 Add tests, fix code!
✗	 False alarm, fix tests!	

Tests accuracy means how much tests outcome matches code correctness.

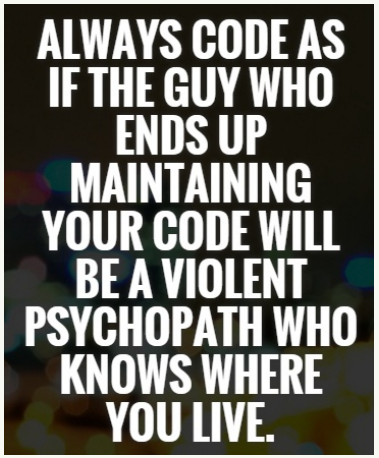
Tests accuracy and reacting to failures

Did tests pass? \ Is code correct?	✓	✗
✓	 Ship it!	 Add tests, fix code!
✗	 False alarm, fix tests!	 Success story!

Tests accuracy means how much tests outcome matches code correctness.

Tests, tests, tests

Improving tests and adding more



0. You won't work on your code! → 
1. Review the existing tests:
   - ↳ Does one failure trigger others?
   - ↳ Is the tested result always the same?
   - ↳ Do you rely on implementation details in tests?
   - ↳ How do you set up the test environment?
   - ↳ Are your tests well documenting how to use your code?
2. Add some more tests
3. Stage, commit and push your work!

Break

General good practices

DRY

Don't Repeat Yourself → The DRY principle

- If you are doing Copy & Paste, you are doing it wrong!
- One of the most basic and yet most disregarded principle
- Code duplication makes the code less maintainable and easy to break

Don't Repeat Yourself → The DRY principle

- If you are doing Copy & Paste, you are doing it wrong!
- One of the most basic and yet most disregarded principle
- Code duplication makes the code less maintainable and easy to break

Trivial example

```
double sum1=0;
for(const auto& data : dataSet1)
    sum1 += data;
sum1 /= dataSet1.size();


double sum2=0;
for(const auto& data : dataSet2)
    sum2 += data;
sum2 /= dataSet2.size();
```


Don't Repeat Yourself → The DRY principle

- If you are doing Copy & Paste, you are doing it wrong!
- One of the most basic and yet most disregarded principle
- Code duplication makes the code less maintainable and easy to break

Trivial example

```
double calculateAverage(const std::vector<double>& dataSet)
{
    double sum=0;
    for(const auto& data : dataSet)
        sum += data;
    return sum / dataSet.size();
}

calculateAverage(dataSet1);
calculateAverage(dataSet2);
```


Don't Repeat Yourself → The DRY principle

- If you are doing Copy & Paste, you are doing it wrong!
- One of the most basic and yet most disregarded principle
- Code duplication makes the code less maintainable and easy to break

Less trivial example

```
double calculateAverage(const std::vector<double>& dataSet)
{
    // ...
    throw std::runtime_error("Error in \"calculateAverage\"");
}
```


Don't Repeat Yourself → The DRY principle

- If you are doing Copy & Paste, you are doing it wrong!
- One of the most basic and yet most disregarded principle
- Code duplication makes the code less maintainable and easy to break

Less trivial example

```
double calculateAverage(const std::vector<double>& dataSet)
{
    // ...
    throw std::runtime_error("Error in \"calculateAverage\"");
}
double calculateAverage(const std::vector<double>& dataSet)
{
    // ...
    using namespace std::string_literals;
    throw std::runtime_error("Error in \""s + __func__ + "\""s);
}
```


__func__ in C++ is an implementation-defined string and is just one among other possibilities.

Don't Repeat Yourself → The DRY principle

- If you are doing Copy & Paste, you are doing it wrong!
- One of the most basic and yet most disregarded principle
- Code duplication makes the code less maintainable and easy to break

Traversing a complex structure

```
// Imagine to have to do this very often
for(auto& line : wiki)
{
    if(line.isHeader())
        continue;
    for(auto& letter : line)
    {
        if(letter.isVowel())
        capitalize(letter);
    }
}
// How can you avoid duplication?
```


Don't Repeat Yourself → The DRY principle

- If you are doing Copy & Paste, you are doing it wrong!
- One of the most basic and yet most disregarded principle
- Code duplication makes the code less maintainable and easy to break

Traversing a complex structure

```
void traverseAndTransform(WikiPage& wiki,
                          std::function<void(Letter)> transform)
{
    for(auto& line : wiki){
        if(line.isHeader())
            continue;
        for(auto& letter : line)
            transform(letter);
    }
}

traverseAndTransform(wiki, /*lambda function*/);
```


KISS

Keep It Simple, Stupid → The KISS principle

Avoid complexity!

Everything should be done as simple as possible, but not simpler.

A. Einstein

- If you have a solution, ask yourself if there is an easier one
- Complexity reduces readability and maintainability
- Use libraries!
- Do not feel cool because you did something hard in a tricky way.
  Feel cool when you did the same in a trivial-to-understand way!
- If you really need something complicated (do you!?), then be sure everybody understands!

Keep It Simple, Stupid → The KISS principle

Simple example

```
std::vector<int> data = {1, -3, 2, 8, 4, -9};

auto first = data.begin(), last = data.end();
while (first != last) {
    if (*first == 4) { // Don't hard-code values!
        makeReportForUser();
        break;
    }
    ++first;
}
```


Keep It Simple, Stupid → The KISS principle

Simple example

```
std::vector<int> data = {1, -3, 2, 8, 4, -9};

auto it = std::find(data.begin(), data.end(), FAILURE);
if(it != data.end())
    makeReportForUser();
```


Keep It Simple, Stupid → The KISS principle

Simple example

```
std::vector<int> data = {1, -3, 2, 8, 4, -9};

//Even better
if(isElementInVector(data, FAILURE))
    makeReportForUser();
```

```
template<typename T>
bool isElementInVector(const std::vector<T>& data,
                       const T& value)
{
    auto it = std::find(data.begin(), data.end(), value);
    return it != data.end();
}
```


Keep It Simple, Stupid → The KISS principle

Example from real life (Quake III Arena, 1999)

Keep It Simple, Stupid → The KISS principle

Example from real life (Quake III Arena, 1999)

```
float Q_rsqrt(float number)
{
    long i;  float x2, y;  const float threehalfs = 1.5F;

    x2 = number * 0.5F;  y  = number;
    // evil floating point bit level hacking
    i  = * (long*) &y;
    i  = 0x5f3759df - (i >> 1);  // what the fuck?
    y  = * (float*) &i;
    // 1st iteration
    y  = y * (threehalfs - (x2 * y * y));
    // 2nd iteration, this can be removed
    // y  = y * (threehalfs - (x2 * y * y));
    return y;
}
```


Keep It Simple, Stupid → The KISS principle

A possible refactoring

Keep It Simple, Stupid → The KISS principle

A possible refactoring

```
float Q_fastInverseSqrt(float number)
{
    float approximateResult
        = calculateApproximateInverseSqrt(number);
    return refineResult(approximateResult, number);
}
```


Keep It Simple, Stupid → The KISS principle

A possible refactoring

```
float Q_fastInverseSqrt(float number)
{
    float approximateResult
        = calculateApproximateInverseSqrt(number);
    return refineResult(approximateResult, number);
}
```

```
float calculateApproximateInverseSqrt(float number)
{
    //Here some very low level operations are carried out.
    //Read here for a detailed explanation.
    long floatAsInteger = * (long*) &number;
    floatAsInteger = 0x5f3759df - (floatAsInteger >> 1);
    return * (float*) &i;
}
```

Keep It Simple, Stupid → The KISS principle

A possible refactoring

```
float Q_fastInverseSqrt(float number)
{
    float approximateResult
        = calculateApproximateInverseSqrt(number);
    return refineResult(approximateResult, number);
}
```

```
float refineResult(float result, float initialNumber)
{
    //Here one iteration of the Newton's method is used.
    //Read here for a detailed explanation.
    float refinedResult = initialNumber * result * result;
    return (3.0F - refinedResult) * result * 0.5F;
}
```


YAGNI

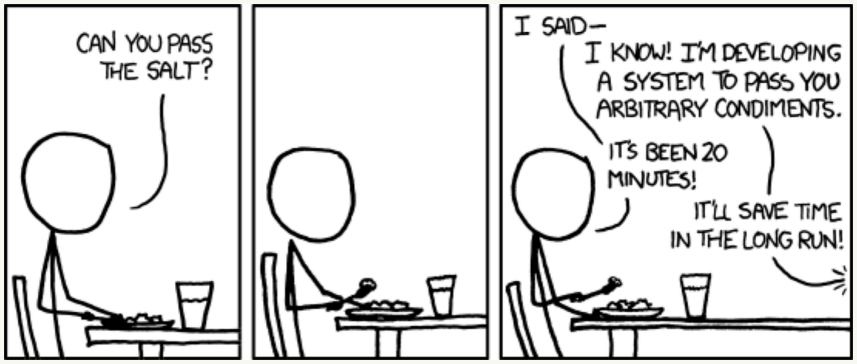
You Aren't Gonna Need It → The YAGNI principle

A possible definition

Always implement things when you actually need them, never when you just foresee that you need them.

Ron Jeffries

- Avoid reasonably unnecessary abstractions
  - ↳ **Yet, it is a judgment call!**
- Maintaining unused code is a burden



Ron Jeffries is a co-founder of extreme programming (XP) philosophy.

You Aren't Gonna Need It → The YAGNI principle

Experience makes you better

> Here's the truth with these patterns/paradigms. A junior dev will ignore them and code something simple that will not scale. A mid level will understand abstraction but over engineer a lot of things because they lack the experience to judge whether they are appropriate in the context of their work. Seniors will go back to simple things and postpone until it's really needed to make it more complex, because life taught them the costs of premature optimizations. The final level is when you understand that all of these are concepts that are good to know but none is a universal truth. [...]
>
> A comment on Reddit

You Aren't Gonna Need It → The YAGNI principle

Experience makes you better

Here's the truth with these patterns/paradigms. A junior dev will ignore them and code something simple that will not scale. A mid level will understand abstraction but over engineer a lot of things because they lack the experience to judge whether they are appropriate in the context of their work. Seniors will go back to simple things and postpone until it's really needed to make it more complex, because life taught them the costs of premature optimizations. The final level is when you understand that all of these are concepts that are good to know but none is a universal truth. [...]

A comment on Reddit

I need to check whether a vector contains an element:

```
// A (too?) specific implementation
bool isElementInVector(const std::vector<double>& data,
                       double value)
{
    auto it = std::find(data.begin(), data.end(), value);
    return it != data.end();
}
```


You Aren't Gonna Need It → The YAGNI principle

Experience makes you better

Here's the truth with these patterns/paradigms. A junior dev will ignore them and code something simple that will not scale. A mid level will understand abstraction but over engineer a lot of things because they lack the experience to judge whether they are appropriate in the context of their work. Seniors will go back to simple things and postpone until it's really needed to make it more complex, because life taught them the costs of premature optimizations. The final level is when you understand that all of these are concepts that are good to know but none is a universal truth. [...]

A comment on Reddit

I need to check whether a vector contains an element:

```cpp
template<typename T>
bool isElementInVector(const std::vector<T>& data,
                       const T& value)
{
    auto it = std::find(data.begin(), data.end(), value);
    return it != data.end();
}
```


You Aren't Gonna Need It → The YAGNI principle

Experience makes you better

Here's the truth with these patterns/paradigms. A junior dev will ignore them and code something simple that will not scale. A mid level will understand abstraction but over engineer a lot of things because they lack the experience to judge whether they are appropriate in the context of their work. Seniors will go back to simple things and postpone until it's really needed to make it more complex, because life taught them the costs of premature optimizations. The final level is when you understand that all of these are concepts that are good to know but none is a universal truth. [...]

A comment on Reddit

I need to check whether a vector contains an element:

```
// Probably NOT straight away!
template<typename Container, typename Item>
bool contains(const Container& container, const Item& value)
{
    // Some implementation possibly calling container
    // member find or ::std::find if container has none
}
```


Optimisation

Do not be disappointed

I have only a couple of slides about optimisation

Correctness comes before performance!

Steps needed seeking for speed:

1. Write **enough** tests
2. Profile the code
3. Benchmark the part you want optimise
4. Change the code
5. Make sure tests still pass
6. `goto` 3 or 2

Do not be disappointed

I have only a couple of slides about optimisation

Correctness comes before performance!

Steps needed seeking for speed:

1. Write **enough** tests
2. Profile the code
3. Benchmark the part you want optimise
4. Change the code ← Optimisation is not just this!
5. Make sure tests still pass
6. `goto` 3 or 2

Beware of optimisations

Premature optimisation

Programmers waste enormous amounts of time thinking about, or worrying about, the speed of noncritical parts of their programs, and these attempts at efficiency actually have a strong negative impact when debugging and maintenance are considered. We should forget about small efficiencies, say about 97% of the time:

Premature optimisation is the root of all evil.

Yet we should not pass up our opportunities in that critical 3%.

D. Knuth

- Compiler do more and more a better job!
- Use optimisations flags (e.g. `-Os`, `-O3` with `g++` and `clang`)
- Never sacrifice clean code in name of **claimed** better performance!
- Only optimise after having used a profiler and having written tests!

Boy scout philosophy

The boy scout rule

Leave the campground cleaner than you found it

Always check in the code cleaner than you checked it out

- To clean an existing code base takes time and requires effort
- Aim for tiny but continuous progress
- Applying the boy scout rule is a way to pay back technical debt
- Any IDE will help you in quickly achieving easy refactoring

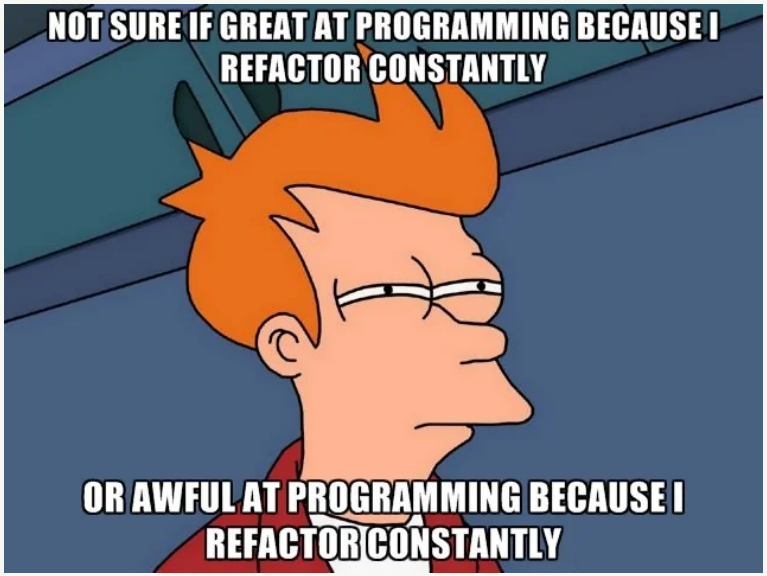
Refactor!

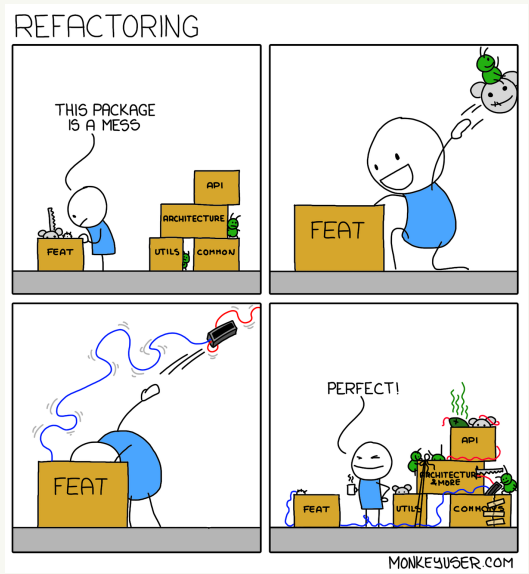
A word about refactoring

A definition

In computer programming and software design, code refactoring is the process of restructuring existing source code without changing its external behaviour. Refactoring is intended to improve the design, structure, and/or implementation of the software, while preserving its functionality.

Wikipedia





Your task

A definition

In computer programming and software design, code refactoring is the process of restructuring existing source code without changing its external behaviour. Refactoring is intended to improve the design, structure, and/or implementation of the software, while preserving its functionality.

Wikipedia

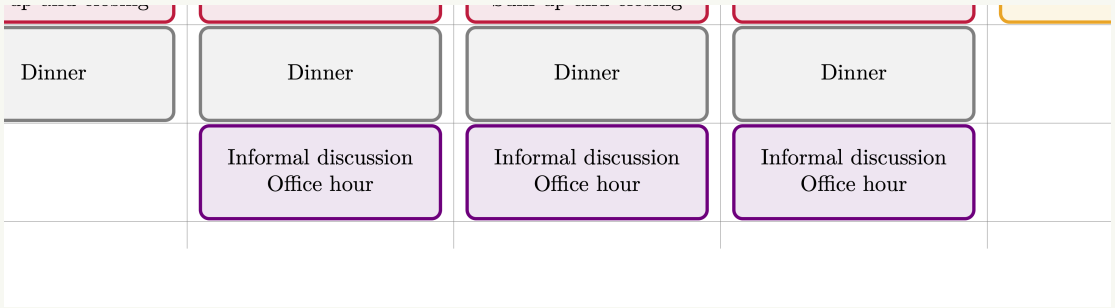
1. Watch out for violations of the learned principles
   - ↳ Any code duplication?
   - ↳ Any part hard to understand?
   - ↳ Any unnecessary generalisation?
2. Improve the code, adding tests first if needed
3. Stage, commit and push your work!

Sum up and closing

Time to reflect on the day

You have an opportunity after dinner!

- ✓ Finish working on exercises in the room
- ✓ Ask further and/or broader questions
- ✓ Enjoy others' discussion



It is not mandatory, but you can use it as you wish.

Time to reflect on the day

1. Take few minutes to look back at the work done:
   - How was the code you received?
   - What's your general impression?
   - Are there unanswered questions?
   - Any challenging aspect?
   - Are you sceptical? If so, about what?
2. Report your group experience/work to the whole class.